

Kubernetes PersistentVolumeClaim Pending Recovery Runbook

Document ID: RB-K8S-005 | Version: 1.0 | Last Updated: 2024-11-15

Category	kubernetes	Severity	P2 - High
Domain	Container Orchestration	Est. Duration	20-35 minutes
Tags	kubernetes, pvc, persistentvolume, storage, pending, storageclass		

Overview

A PersistentVolumeClaim stuck in Pending state means Kubernetes cannot bind a PersistentVolume to satisfy the claim. Any pod that mounts this PVC will also be stuck in Pending. Common causes: no matching StorageClass, insufficient storage capacity, access mode mismatch, or missing CSI driver.

Prerequisites

- kubectl with cluster-admin or storage-admin permissions
- Access to the cloud provider console (AWS/GCP/Azure) for volume verification
- Knowledge of the StorageClass provisioner in use

Response Steps

Step 1: Identify the Pending PVC

List all PVCs in the namespace and confirm which ones are stuck in Pending state.

```
kubectl get pvc -n
kubectl describe pvc -n
```

NOTE: The Events section of describe output reveals the exact binding failure reason.

Step 2: Check StorageClass Exists

Verify the StorageClass referenced in the PVC spec exists in the cluster.

```
kubectl get storageclass
kubectl describe storageclass
```

Step 3: Check Available PersistentVolumes

Look for unbound PVs that match the PVC's access mode, storage class, and capacity.

```
kubectl get pv
kubectl get pv -o custom-columns='NAME:.metadata.name,CAPACITY:.spec.capacity.storage,ACCESS:.spec.accessModes,STATUS:.status.phase,CLAIM:.spec.claimRef.name'
```

Step 4: Check CSI Driver Status

If using a CSI driver (EBS, GCE PD, Azure Disk), verify the CSI pods are running in kube-system.

```
kubectl get pods -n kube-system | grep csi
kubectl describe pod -n kube-system
```

Step 5: Check Node Affinity Constraints

If the PVC has a WaitForFirstConsumer binding mode, it binds only when a pod is scheduled. Check pod scheduling constraints.

```
kubectl get storageclass -o jsonpath='{.volumeBindingMode}'
kubectl get pods -n | grep Pending
kubectl describe pod -n | grep -A10 'Events'
```

NOTE: WaitForFirstConsumer is normal — the PVC binds when the pod is scheduled to a node.

Step 6: Manually Create PV if Dynamic Provisioning Fails

If the provisioner is not creating PVS, manually create a PV that matches the PVC requirements.

```
# Create a PV manifest and apply:
kubectl apply -f pv-manual.yaml
kubectl get pvc -n
```

NOTE: Manual PVs must exactly match: storageClassName, accessModes, capacity, and volumeMode.

Rollback Steps

Rollback Step 1: Delete and Recreate PVC

If the PVC spec was misconfigured, delete it and recreate with corrected StorageClass or capacity.

```
kubectl delete pvc -n
kubectl apply -f corrected-pvc.yaml
```

Rollback Step 2: Restore from Volume Snapshot

If data loss occurred, restore the PVC from the most recent VolumeSnapshot.

```
kubectl get volumesnapshot -n
kubectl apply -f restore-from-snapshot.yaml
```

Step Dependencies

Step	Depends On	Can Parallelize With
Step 1: Identify Pending PVC	—	—
Step 2: Check StorageClass	Step 1	Step 3
Step 3: Check Available PVs	Step 1	Step 2
Step 4: Check CSI Driver	Step 1	Step 2, Step 3
Step 5: Check Node Affinity	Step 2	Step 4
Step 6: Manually Create PV	Step 2, Step 3, Step 4	—

Maintained by Platform Engineering. For updates ping #platform-oncall.